

Architecture, runbook et garde-fous du code Aster.

Ce document complete le cockpit de resultats. Le cockpit dit quelles lanes performent. Cette page dit quels fichiers font quoi, lesquels lancer, lesquels ne pas toucher, et comment continuer sans se perdre dans les modules Python.

Decision documentaire

On garde deux HTML principaux.

`core-equity-aster-research-consolidated-report.html` pour les resultats, et ce fichier pour l'architecture/runbook. Les details longs restent en Markdown.

Runners actifs recommandes

4

Doc source principale

README

Rapport resultats

HTML

Execution reelle

0

Navigation

Decision Structure Modules Commandes Artefacts Garde-fous Prochaines etapes

Decision: ne pas tout mettre dans un seul HTML

1. Resultats

HTML principal

`core-equity-aster-research-consolidated-report.html` reste le cockpit de performance: ROI, WR, PF, mark/index, WS quality, exchangeInfo.

2. Code et runbook

ce fichier

`core-equity-aster-code-map.html` explique le code, les commandes et ce qu'il faut ou non lancer.

3. Registry strategies

identite

`core-equity-aster-strategy-registry.html` dit quel script Python correspond a quelle famille de strategie exacte.

4. Memoire longue

Markdown

Les details exhaustifs restent dans `aster-working-map.md`, `aster-official-docs-gap-audit.md` et les archives.

Structure actuelle du dossier Aster

Zone	Role	Regle
<code>data_sources/</code>	Sources publiques Aster: univers, WS, validation endpoints, qualite symbole.	Read-only uniquement. Les nouveaux modules data doivent aller ici.
<code>research/</code>	Rapports d'aide a la decision: dual-track, validation champions, status runners, v2 plan.	Lit les artefacts locaux. Ne doit pas relancer de gros backtests sauf runner explicitement dedie.
Racine <code>aster/</code>	Runners historiques, backtests, paper trading, adapters, rapports principaux.	Stable pour compatibilite. Ne pas deplacer pendant que les terminaux tournent.
<code>archive/research_artifacts/</code>	Anciens CSV, legacy, snapshots historiques.	Conserver pour preuve, ne pas utiliser comme source prioritaire.

Modules a connaitre

Backtests / discovery

`aster_strategy_registry.py` : registry canonique scripts/familles/statuts. A lire avant d'ajouter un nouveau runner.

`backtest_strategy_discovery.py` : runner principal multi-symboles / multi-timeframes. Produit les CSV V2 tagges.

`backtest_aster_v2.py` : wrapper v2 unifie, construit un univers plus intelligent via plan/preflight.

`backtest_reality_checked.py` : relance discovery puis filtre avec le reality check public.

Modele perps

`aster_perps_model.py` : frais officiels, funding public, cache `exchangeInfo`, verdict de filtres exchange.

`aster_mark_index_replay_filter.py` : compare last/mark/index pour eviter les faux edges.

`aster_microstructure_replay_validator.py` : verifie si les champions auraient ete remplissables via depth + aggTrades.

`aster_mcp_market_data_adapter.py` : normalise les endpoints publics Aster V3.

Data sources

`data_sources/source_validation.py` : prouve quels endpoints sont utilisables.

`data_sources/public_universe_snapshot.py` : construit l'univers public Aster.

`data_sources/ws_symbol_quality_report.py` : qualite WS par symbole.

Rapports

`aster_research_consolidated_report.py` : genere le cockpit HTML principal.

`research/exchangeinfo_run_status.py` : lit les heartbeats des runs exchangeInfo.

`research/dual_track_research_report.py` : garde exploitation et exploration separees.

Commandes propres a lancer

Suivre les runs exchangeInfo en cours

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.research.exchangeinfo_run_status
```

Regenerer le cockpit resultats

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_research_consolidated_report
```

Verifier la microstructure des champions

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_microstructure_replay_validator --champions LABUSD:5h:long,INTCUSD:30m:long,CRCLUSD:2h:long
```

Ce filtre est read-only: il lit `aggTrades` / `historicalTrades` et `depth`, mesure spread, slippage, gaps, volume reel et nombre de trades, puis retourne `microstructure_ok`, `thin_liquidity`, `gap_risk` ou `not_enough_trades`.

Suivre les lanes promotion-ready en forward paper

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_promotion_ready_lanes_report --forward-cycle --run-forever --sleep-seconds 300 --forward-window-trades 1000 --p
```

Ce runner ne suit que les lanes `PROMOTION_READY`. Il ne cree aucun trade reel: `--persist` ecrit seulement dans le ledger paper-trading.

Diagnostiquer la microstructure au moment des entrees

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_promotion_ready_lanes_report --entry-micro-diagnostic --forward-window-trades 1000
```

Rejoue les lanes `PROMOTION_READY`, extrait les entrees, puis verifie le flux `aggTrades` autour du timestamp d'entree. Limite: pas de carnet historique public, donc le test valide la liquidite trade-tape, pas le spread historique exact.

Post-mortem strict du ledger promotion-ready

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_promotion_ready_lanes_report --strict-ledger-postmortem --limit-pairs 60
```

Lit les sorties paper persistees, rattache chaque sortie a son entree, puis teste si l'entree aurait passe des garde-fous plus stricts: au moins 50 trades, au moins 20k USD de volume quote et aucun gap superieur a 30 secondes dans la fenetre d'entree. Dernier verdict: `strict_filter_reduces_damage`, donc le filtre reduit la perte mais ne valide pas encore LAB en forward.

Lancer le monitor strict promotion-ready

```
Set-Location D:\trading-agent\backend
python -m services.onchain.aster.aster_promotion_ready_lanes_report --strict-forward-cycle --run-forever --sleep-seconds 300 --forward-window-trades 1000
```

Variante separee du monitor promotion-ready historique. Elle refuse les entrees dont la fenetre trade-tape est trop mince et ecrit dans

`paper_trading_strict_promotion_ready_forward_monitor.csv` avec un `event_type` distinct.

Relancer le RAG documentaire

```
Set-Location D:\trading-agent
D:\trading-agent\venv-rag\Scripts\python.exe D:\trading-agent\rag\scripts\rag_tool.py ingest --domain memory
```

Les 4 terminaux H24 actuels

Les tags propres sont `core_exchangeinfo`, `macro_equity_exchangeinfo`, `equities_explore_exchangeinfo`, et `aster_v2_exchangeinfo`. Ne pas ajouter dix terminaux de plus tant que ces quatre n'ont pas donne au moins un cycle complet.

Artefacts importants

Artefact	Usage	Lecture
<code>paper_trading_strategy_discovery_*_v2.csv</code>	Resultats append-only par univers/tag.	Source principale des backtests.
<code>*_heartbeat.json</code>	Etat du runner pendant qu'il tourne.	Utile avant que le CSV soit ecrit.
<code>aster_public_exchange_info_cache.json</code>	Cache public des contraintes Aster par symbole.	Alimente les champs <code>exchange_filter_*</code> .
<code>aster_research_consolidated_report_latest.json</code>	Version machine du cockpit HTML.	Utile pour analyses rapides sans parser HTML.
<code>aster_exchangeinfo_run_status_latest.json</code>	Status local des quatre runs exchangeInfo.	Dit si les CSV sont prêts.
<code>aster_microstructure_replay_validator_latest.json</code>	Validation fillability des champions via trades reels + carnet.	Filtre de promotion avant forward/paper trading.
<code>paper_trading_promotion_ready_forward_monitor.csv</code>	Suivi forward des lanes promotion-ready uniquement.	Track record virtuel cible.
<code>aster_entry_microstructure_diagnostic_latest.json</code>	Contrôle anti-overfit des entrees replay via aggTrades autour du timestamp.	Filtre supplementaire avant validation forward.

Garde-fous non negociables

aucun trade reel aucun wallet order aucune API key trading pas de refactor pendant runners actifs read-only public first

Les endpoints signes Aster comme `income`, `leverageBracket`, `adiQuantile`, `commissionRate`, MMP, strategy orders ou transfers sont connus, documentes, mais bloques tant qu'une policy credentials read-only/testnet n'existe pas.

Ce qu'il reste a faire

Maintenant

Attendre les premiers CSV `*_exchangeinfo_v2.csv`, puis analyser les champions avec les nouveaux champs exchange.

Ensuite

Construire le validateur microstructure `aggTrades/historicalTrades` pour rejouer les champions avec slippage/fill plus fin.

Plus tard

Ajouter funding par position, session filters equities/macro, puis eventuellement policy read-only/testnet pour brackets reels.